Azure SQL Elastic Pools

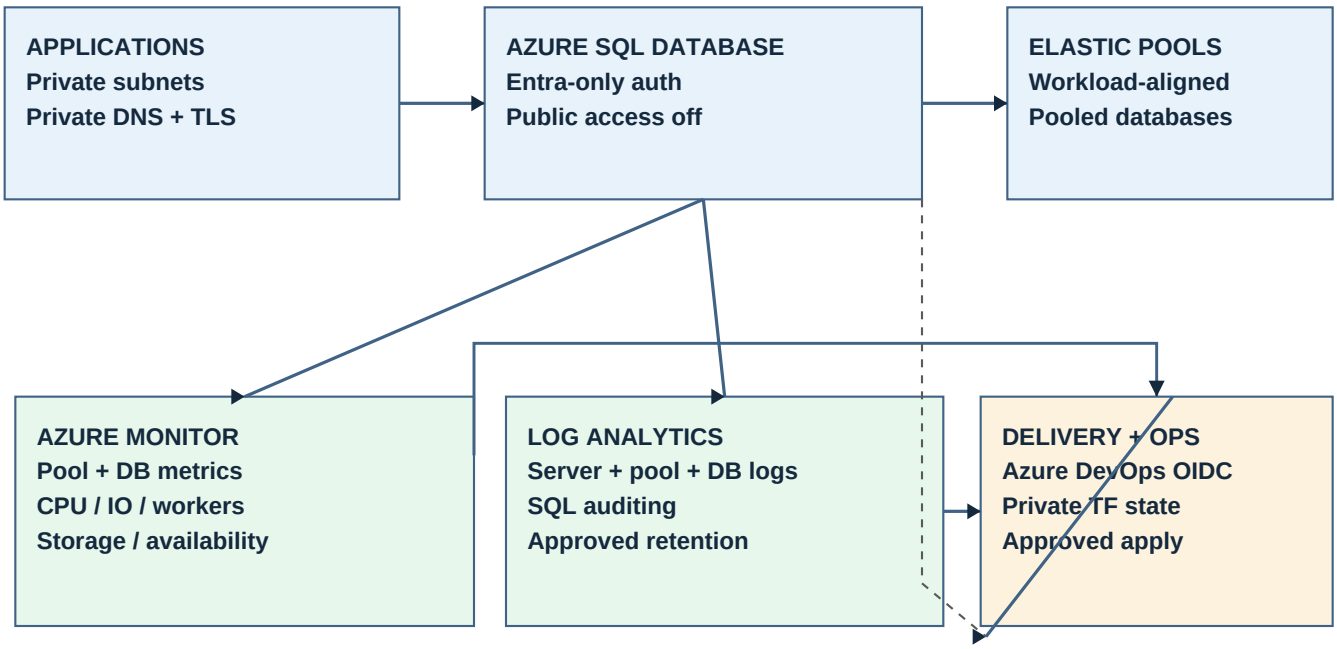
DevOps Planning & Implementation

Readiness plan, environment worksheet, Terraform,
Azure DevOps, operations, and recovery

Reference scale: 10+ elastic pools / 100+ databases

Planning template only - no Azure subscription or deployment included.

PRODUCTION REFERENCE ARCHITECTURE



Applications reach the Entra-only SQL server over Private Link and private DNS. Terraform is delivered by Azure DevOps workload identity; its private Blob state backend uses lease locking. Azure Monitor metrics, server auditing, and resource diagnostics feed Log Analytics and staffed alerts. A separately tested recovery region supports the approved continuity plan.

Reference only, not a deployed environment. Confirm topology, regional redundancy, product support, SKU, cost, and RPO/RTO with service owners.

Azure SQL elastic pool production readiness plan

Document control

Field | Value
Status | Planning baseline; environment-specific decisions are intentionally pending.
Scope | Azure SQL Database elastic pools, Terraform, private networking, Azure DevOps delivery, monitoring, operations, and recovery.
Cost boundary | No Azure resource is created by this plan. Repository and credential-free document/code checks are designed to be cost-free. Later Azure plans/applies and deployed services may incur charges.
Authority to deploy | None is granted by this document. A separately approved change and authorized subscription are required.

Required companion | Complete every applicable row in environment inputs and the deployment runbook.

1. Outcomes and exclusions

Required outcomes

- A workload-measured and cost-reviewed Azure SQL pool design with named service and database owners.
- A private-only SQL access path, Entra authentication, approved identity/RBAC, and governed secrets handling.
- A secured remote Terraform state backend, reproducible provider lock file, protected source branch, and federated Azure DevOps identity.
- A reviewed plan artifact, server-side protected production approval, and a controlled apply of that exact plan.
- Measurable monitoring, tested alert routing, backup/restore evidence, and recovery procedures tied to approved RPO/RTO.
- An operations handover with ownership, dashboards, escalation paths, runbooks, and scheduled reviews.

Explicit exclusions until separately designed and approved

- Selecting an actual subscription, tenant, region, SKU, pool size, budget, data classification, or production owner.
- Creating or changing any Azure resource, role assignment, Azure DevOps resource, DNS record, or network rule.
- Schema/data migration, application code changes, identity enrollment, data import, and production cutover.
- Grafana/Prometheus platform integration, exporter or scrape configuration, and dashboard implementation; track and approve this as a separate observability task.
- A claim that regional failover, restore, high availability, capacity, or service-level objectives have been tested.
- Secrets, personal information, credentials, subscription IDs, customer data, or production state in this Git repository.

The Terraform root is a starting implementation boundary, not proof that every organization policy or Azure feature requirement is met. Review it against current Microsoft documentation, current provider schemas, the landing zone, and security policies before any later deployment.

2. Decision principles

1. No guessed production values. Workload owners and platform owners fill in the companion worksheet with evidence and approvals.
2. Fail closed. Missing prerequisites, unexpected plan actions, unknown cost, network ambiguity, or an untested recovery objective stop the gate.
3. Separate planning from execution. Mermaid diagrams, Markdown, Terraform formatting, PDF generation, and source review do not deploy Azure resources. A Terraform plan requires real credentials and may access the Azure control plane; run it only after the plan gate.
4. One owner per decision. Each decision has a named accountable role, evidence, and approval record.
5. Least privilege and protected execution. Federated identity, scoped RBAC, server-side Azure DevOps branch controls, protected environment approvals, and exact-plan apply are mandatory.
6. Test recoverability, not just configuration. A configured retention value is not restore evidence; a DR diagram is not a failover test.
7. Track ongoing cost and operational burden. Estimate compute, storage, backups, monitoring ingestion/retention, network, DR capacity, and support before approval.

3. Delivery stages and gates

No stage assumes a calendar duration; the accountable owner schedules it after dependencies and change windows are known.

Stage | Work and deliverables | Accountable owner | Exit gate
0. Scope and service ownership | Service charter; supported use cases; application/database inventory; criticality; owners; data residency/classification; business hours and support model. | Service owner | Every database has a business and technical owner; scope and exclusions approved.
1. Workload discovery | Peak and percentile CPU, data/log I/O, workers, sessions, storage, concurrency,

latency, seasonality, batch/reporting peaks, growth and isolation needs. | DBA + application owners | Representative evidence covers normal, peak, burst, and growth scenarios; workload classes reviewed.
2. Architecture and recovery decisions | Primary/secondary regions, topology, endpoint/DNS paths, SKU and feature selection, per-database limits, backup, RPO/RTO, failover/restore method, retention and compliance. | Platform architect + DBA + security | Architecture, recovery objectives, exceptions, and options analysis approved.
3. Cost and capacity approval | SKU/region availability, quotas, capacity headroom, licensing eligibility, Azure pricing estimate, logging volume/retention, backup and DR cost, budget/alerts. | FinOps + capacity owner | Written estimate and quota evidence approved; budget owner accepts cost exposure.
4. Landing-zone readiness | Subscription/tenant ownership, resource groups, policy/provider registration, network/subnet, private DNS, routes/NSGs, state account, private DNS, RBAC and monitoring workspace decisions. | Cloud platform + network | All IDs and access evidence collected; private resolution and private state path tested by the approved operators.
5. Security and compliance review | Threat model, identity/data access, private access, encryption, auditing, Defender/Policy controls, least privilege, state/plan sensitivity, retention, incident contact. | Security + compliance | Controls mapped to requirements; risks and exceptions have owners and approvals.
6. Delivery platform readiness | Repository protections, PR reviewers, CI, provider mirror or registry access, Linux agent, OIDC service connection, backend variables, protected environment, branch checks. | DevOps + cloud platform | No secrets in source; federation and branch/environment checks verified; nonproduction path works.
7. Nonproduction implementation | Fill nonproduction values; commit provider lock file; offline validation; authenticated nonproduction plan and apply; app connection, monitoring, load, restore and runbook tests. | IaC owner + DBA + app owner | Change approved; tests pass; issues fixed; artifacts and evidence retained.
8. Production readiness review | Complete inputs, cost, capacity, security, network, access, plan, operational and recovery evidence; freeze unrelated state changes. | Change manager | Go/no-go meeting records approver, window, rollback/stop decision owner, and exact reviewed artifact.
9. Production deployment | Obtain a fresh production plan; verify target and plan; approve protected environment; apply only the exact approved plan; monitor changes. | Change manager + platform operator | Deployment matches expected actions; no unresolved warnings or unplanned replacement/deletion.
10. Acceptance and handover | Smoke test through private DNS, Entra access, app health, pool/database metrics, audit/logs, alert delivery, inventory, cost, backup/restore and DR evidence. | Service owner + operations | Signed acceptance; on-call and escalation ownership active; open risks formally accepted.
11. Operate and improve | Capacity/cost/security/alert review, patch/service health, access review, restore and failover drills, Terraform drift/change review, lifecycle updates. | Service owner + SRE/DBA | Scheduled cadence, evidence store, action owners, and re-approval triggers established.

Stop/go checklist

Do not pass a gate if any applicable item is unknown, unowned, or unsupported:

- [ ] Correct tenant, subscription, region, policies, quotas, and provider registrations are confirmed by the platform owner.
- [ ] Workload placement and capacity are based on evidence, not sample values or database count.
- [ ] SQL, VNet/subnet, Private DNS, state-storage, and agent network paths are identified and tested.
- [ ] Entra administrator and application identities are approved; permissions are scoped and reviewed.
- [ ] Cost estimate includes compute, storage, backup, diagnostics/Log Analytics, private networking, and recovery-region capacity.
- [ ] Backup, retention, restore, RPO/RTO, and regional recovery decisions are approved and testable.
- [ ] Terraform version/provider lock file, repository protection, state security, OIDC, branch check and environment approval are ready.
- [ ] The exact plan has no unapproved replacement, delete, public-access change, or permission escalation.
- [ ] Change window, communications, rollback/stop authority, on-call, and acceptance evidence owners are recorded.

4. Work breakdown and artifacts

Workstream | Required repository or controlled-system artifacts | Evidence retained outside Git
Product/service definition | Service inventory schema, ownership model, supported workload classes. | Approved inventory, service tier and change record.
Architecture | architecture.md, architecture diagram, deployment flow, and recovery flow. | Network approval, regional feature support, threat model and decision record.

Environment configuration | Non-secret, reviewed environment configuration derived from environment-inputs.md. | Protected tfvars/variable group values and approval history; secrets only in approved secret store.
Infrastructure code | terraform/production/*.tf, reviewed module/provider versions, generated .terraform.lock.hcl, and code review. | Provider checksums, plan approval, CI results and Azure activity evidence.
Delivery | azure-pipelines.yml, scripts/terraform-ado.sh, protected branch, workload identity, restricted agent, gated environment. | Azure DevOps service connection, access review, branch checks, approvals and artifact permissions.
Operations/observability | Built-in Azure-native Log Analytics diagnostics, SQL auditing, six per-pool saturation alerts, action group, and Azure Monitor workbook; application telemetry contract and operating procedures. Grafana/Prometheus integration and dashboard setup are a separately scoped follow-up, not part of this delivery. | Ingestion freshness/category review, workbook query checks, alert routing test, on-call records, incident exercises and DBA/SRE threshold tuning. For the separate Grafana/Prometheus task, approve platform/source/security/cost design and verify data coverage, access, dashboards, alert routing, and support ownership.
Resilience | Approved recovery design and environment mapping. | Isolated restore test, failover exercise, measured data loss/RTO and corrective actions.

5. Configuration and release workflow

1. Review this plan, the architecture, current Terraform scope, and the complete operator runbook.
2. Obtain the required named owners and fill the decision/evidence rows in environment-inputs.md. Leave unknown values as TBD; never turn a guess into a Terraform value.
3. Have each owning team verify IDs and permissions in the actual approved tenant/subscription through approved access channels. Do not paste credentials, secrets, or customer data into Git.
4. Establish shared services first: landing-zone policy and providers, VNet/subnet, private DNS, monitoring ownership, state storage/private endpoint, and state RBAC. Verify paths from the intended self-hosted deployment agent.
5. Protect main, require review, configure OIDC, restrict service-connection use to protected refs/heads/main with a server-side branch control check, and configure sql-prod approvals.
6. Create a nonproduction .tfvars in the approved protected configuration store. Review names, supported features, pool mapping, limits, backup settings, tags, and full cost estimate.
7. Run credential-free formatting/validation and repository checks. Provider initialization may need the approved registry/mirror. Review and commit the lock file in a controlled, connected environment.
8. Run nonproduction deployment from an approved runner. Review exact plan actions before apply. Execute application connectivity, load, permissions, monitoring, notification, restore and recovery tests.
9. Close defects and capture evidence. Have service, DBA, network, security, FinOps, operations, and change owners sign off the production gate.
10. At the approved change window, ensure no conflicting Terraform operations, create a fresh plan against the intended backend/workspace, inspect its target and all actions, and publish it with restricted retention/access.
11. Obtain environment approval for that plan. Apply the exact saved plan artifact. If source, inputs, state, or artifact changes, discard the approval and generate a fresh plan.
12. Complete acceptance/handover; retain change evidence in the organization-approved system, not in public logs or the source repository.

See the lifecycle diagram and the operator deployment procedure for detailed preparation and command steps.

6. Cost-free repository work vs. paid Azure deployment

Activity | Azure resource charge expected? | Notes
Edit/review Markdown, Mermaid source, Terraform, YAML, and scripts locally | No | Normal workstation and GitHub usage is outside Azure resource billing.
Render the PDF locally with Node.js | No | Uses local compute; no Azure API calls.
Terraform fmt, Bash syntax check, and static document checks | No | Local operations; no Azure resources.
Terraform init -backend=false | No Azure resource deployment | Downloads provider packages from the configured source; network or third-party registry terms may apply.
Terraform validate after initialization | No Azure resource deployment | Checks configuration against the downloaded provider schema.
Terraform plan with Azure credentials | No resources are intentionally created | It authenticates and queries Azure; it may incur non-resource operational/network costs and is not part of this

credential-free package check. Do not run before the plan gate.
State backend bootstrap | Yes, if implemented | Storage, private endpoint, DNS/network, logs and backup may incur cost. Requires approved subscription and cost review.
SQL Terraform apply | Yes | SQL compute/storage, backups, monitoring ingestion/retention, private networking, and optional DR are billable.
GitHub Actions or Azure DevOps hosted/self-hosted runners | Depends on provider/organization plan and runner model | Review platform quotas/billing separately; Azure resource billing is distinct.

No subscription is provided or assumed. No Azure login, plan, apply, state bootstrap, quota validation, or Azure resource creation is performed by this planning work.

7. Risk register

Risk | Impact | Prevention / response | Owner
Example capacity is mistaken for workload sizing | Saturation, throttling, overspend or outage | Require measured profile, regional SKU/quota check, headroom and signed capacity review. | DBA + workload owner
Shared pool noisy-neighbor effects | Critical database degradation | Classify workload; isolate hot/regulated tenants; test burst and contention. | DBA
Private endpoint/DNS not reachable from app or runner | Application outage or failed plan/apply | Test name resolution and routes from each actual network; never bypass with public access. | Network
State identity or branch controls are too broad | Unauthorized infrastructure modification or state exposure | OIDC, least privilege, protected refs, server-side branch checks, restricted state and artifacts. | Security + DevOps
State loss/corruption or concurrent changes | Resource drift or unsafe changes | Blob leases, versioning/recovery controls, restricted access, state restoration procedure and change freeze. | IaC platform
Backup settings misread as tested recovery | Excessive data loss/downtime | Define RPO/RTO and run isolated restore and regional exercises with evidence. | Service owner + DBA
Diagnostic ingestion/retention cost is underestimated | Unexpected operational spend | Model volume and retention, alert on cost, review ingestion and destination policy. | FinOps + observability
Provider/service behavior changes | Failed plan or incompatible rollout | Pin CLI/provider, commit checksums, review release notes and retest in nonproduction. | IaC owner
Environment values remain placeholders or conflict | Wrong target, invalid deployment, production misconfiguration | Worksheet sign-off; validate subscription/tenant and resource IDs at plan; fail closed. | Release manager
Pipeline YAML modified by untrusted PR | Credential exposure or unauthorized deployment | Server-side service-connection branch checks and protected environments; YAML conditions alone are insufficient. | DevOps/security

8. Change, rollback, and completion evidence

Infrastructure changes are not automatically reversible. A rollback may require a forward Terraform change, restoring data to a separate target, or an application-level recovery. Do not assume terraform destroy is safe or supported. Database resources use deletion protection in the current starter module; any exception must be explicit, backed up, separately approved, and reviewed for replacement/deletion effects.

For every production release retain, in the approved change system:

- Approved service inventory, design decisions, cost estimate, quota/SKU evidence, and owner approvals.
- Security, network, data classification, compliance, and identity/RBAC reviews.
- CI run, commit ID, Terraform/provider lock version, exact plan artifact identifier/checksum and approvals.
- Pre/post deployment health, app connectivity, SQL identity checks, alert delivery, diagnostics, and cost baseline.
- Restore/failover test results, measured RPO/RTO, data integrity checks, unresolved risks and owners.
- Final go/no-go, change communications, acceptance, and handover acknowledgements.

Completion means the acceptance and handover checklist in the operations runbook is satisfied, evidence owners have archived evidence, and all remaining risks have an approver and due date.

9. Review cadence and change triggers

Review this plan and environment worksheet at least annually and whenever any of these changes: region or subscription; data classification or residency; workload/ownership/capacity; RPO/RTO; SKU/provider version; network, DNS, identity, branch protection or service connection; retention or cost policy;

Azure service feature support; or a failed restore, security incident, or material service incident. Revalidate links and Azure service limits against current Microsoft documentation.

Azure SQL production environment inputs and approval worksheet

This worksheet is intentionally unfilled. No Azure subscription, tenant, workload, region, organization, service owner, network, budget, or approval was provided for this repository. TBD is a planning status-not a Terraform value and not authorization to deploy. Never invent values to make the template validate.

Complete it through the approved change/design process. Keep credentials, tokens, passwords, private certificates, customer data, private connection strings, and sensitive topology out of Git and this worksheet. Record only non-secret resource identifiers here if the repository's access classification permits them; otherwise keep the completed copy in the approved restricted configuration system.

Document field | Value
Service/application name | TBD
Document owner / reviewer | TBD
Version / review date | TBD
Change or architecture decision reference | TBD
Status | NOT APPROVED FOR DEPLOYMENT

1. Scope, ownership, and impact

Input | Approved value / evidence reference | Accountable approver
Business service and application(s) | TBD | Service owner
Business criticality and user impact | TBD | Service owner
Database inventory and source systems | TBD | Application owner + DBA
Database count, growth and onboarding sequence | TBD | DBA
Database/application technical owner per item | TBD | Application owner
Service hours, support model and on-call rotation | TBD | Operations
Planned maintenance windows and prohibited dates | TBD | Service owner
Data residency, classification and regulatory boundary | TBD | Security/compliance
Service dependencies and downstream consumers | TBD | Application owner
Change owner, implementer, independent reviewer | TBD | Change manager
Go/no-go authority and stop/rollback decision owner | TBD | Change manager

2. Tenant, subscription, location, and resource scope

Input | Approved value / evidence reference | Accountable approver
Entra tenant ID (GUID) | TBD | Identity/platform
Azure subscription ID (GUID) | TBD | Subscription owner
Subscription display name / management group | TBD | Cloud platform
Primary region and allowed paired/recovery region | TBD | Architecture + compliance
Application resource group name and owner | TBD | Cloud platform
Resource naming convention and uniqueness check | TBD | Cloud platform
Required resource providers registered | TBD | Subscription owner
Azure Policy assignments/exemptions | TBD | Governance
Region/SKU quota evidence and quota request reference | TBD | Capacity owner
Deployment identity's exact RBAC roles and scopes | TBD | Cloud security
Budget, cost center and alert recipient | TBD | FinOps

3. Workload discovery and pool design

Attach a protected inventory/profile for each workload class. Do not put customer queries, row data, PII, or secrets in this repository.

Profile field | Evidence/status
Collection period and representativeness (including peak/seasonal events) | TBD
Per-database size now, growth rate, and forecast horizon | TBD
Peak/p95/p99 CPU, data I/O, log I/O, workers, sessions, concurrency | TBD

Query latency/throughput objectives and observed baselines | TBD
Batch, month-end, reporting, ETL and deployment peaks | TBD
Workload classes, noisy-neighbor analysis and isolation needs | TBD
Database-to-pool mapping and owner for every database | TBD
vCore/DTU model and service tier decision with feature rationale | TBD
SKU name, hardware family and license eligibility evidence | TBD
Per-pool capacity, storage cap, and headroom basis | TBD
Per-database minimum/maximum resource bounds and fairness rationale | TBD
Zone redundancy decision and region/SKU support evidence | TBD
Elastic pool/database limits and Azure service-limit review | TBD
Nonproduction load-test plan/results and acceptance criteria | TBD
Expected growth/capacity review triggers | TBD

Pool inventory to be approved

Pool key | Workload class | Region | Tier / SKU / family | Capacity and storage | Database map and limits | Zone redundant? | Evidence / approval
TBD | TBD | TBD | TBD | TBD | TBD | TBD | TBD

Repeat the row for every pool. Use real Terraform keys only after owners approve the mapping and measurements.

4. Network, private DNS, and name resolution

Input | Approved value / evidence reference | Accountable approver
Hub/spoke or landing-zone architecture reference | TBD | Network architect
VNet and region | TBD | Network
Private Endpoint subnet resource ID and available IP capacity | TBD | Network
Subnet Private Endpoint policy/NSG/route requirements | TBD | Network/security
Existing Private DNS zone privatelink.database.windows.net resource ID | TBD | DNS owner
Private DNS VNet links and resolver/forwarding design | TBD | DNS owner
Application subnet(s), routes and TCP 1433 allowance | TBD | Network/security
Private endpoint approval/role requirement | TBD | Network/platform
Private DNS resolution test from each application path | TBD | App/network
Private DNS and TCP test from deployment agent | TBD | DevOps/network
Explicit statement that public SQL access remains disabled | TBD | Security

5. Identity, security, and compliance

Input | Approved value / evidence reference | Accountable approver
Entra SQL administrator group display name | TBD | Identity/DBA
Entra SQL administrator group object ID (non-zero GUID) | TBD | Identity
Tenant match verified for that group | TBD | Identity
Application managed identities / Entra groups and database roles | TBD | App owner + DBA
Human DBA operator groups, PIM and access-review cadence | TBD | Identity/security
Azure DevOps service connection name and federated subject | TBD | DevOps/identity
Exact deployment RBAC actions and resource scopes | TBD | Cloud security
Private state backend data-plane RBAC principal/scope | TBD | IaC platform
TLS/client driver requirements and connection validation | TBD | Security/app
TDE/platform encryption and key-management requirement | TBD | Security/compliance
Defender for SQL and vulnerability/security monitoring policy | TBD | Security
Auditing destination, access, retention and review owner | TBD | Security/compliance
Data deletion, legal hold, restore isolation and evidence retention | TBD | Compliance/DBA
Threat model, risk acceptance and exception record | TBD | Security

Never add a SQL administrator password. The current Terraform template enables Microsoft Entra-only authentication; confirm that this is compatible with all approved clients before deployment.

6. Backup, recovery, and continuity

Input | Approved value / evidence reference | Accountable approver
RPO per service/workload class | TBD | Service owner
RTO per service/workload class | TBD | Service owner
Point-in-time restore retention per database | TBD | DBA/compliance
Backup redundancy setting and regional residency approval | TBD | DBA/compliance
Long-term retention / legal hold requirements and implementation | TBD | Compliance
Recovery region, subscription, quota and secondary pool sizing | TBD | Platform + DBA
Geo-replication/failover-group topology and supported feature review | TBD | DBA/architecture
Listener, private DNS, endpoint and app retry behavior after failover | TBD | App/network

Restore target isolation and required access approvals | TBD | Security/DBA
Restore test date, dataset class, integrity checks and measured RTO | TBD | DBA
Planned and forced failover authority and criteria | TBD | Incident commander
Recurring recovery drill cadence and evidence owner | TBD | Service owner

7. State backend and delivery platform

Input | Approved value / evidence reference | Accountable approver
State resource group, subscription and region | TBD | IaC platform
Globally unique StorageV2 account name / redundancy | TBD | IaC platform/FinOps
Blob container and unique, stable state key | TBD | IaC platform
Private Blob endpoint, DNS zone/link and agent reachability | TBD | Network
Public access disabled, shared key disabled, TLS policy | TBD | Security
Versioning, deletion recovery, lock and backup/restore policy | TBD | IaC platform
State access groups, pipeline data-plane role and scope | TBD | Cloud security
Azure DevOps project, repo and protected main rules | TBD | DevOps
Linux agent pool, owner, patching, egress and private DNS | TBD | DevOps
Terraform CLI version and approved provider registry/mirror | TBD | IaC platform
Provider version and committed .terraform.lock.hcl review | TBD | IaC reviewer
Variable group name and authorized pipelines | TBD | DevOps
sql-prod environment approvers/checks and access controls | TBD | Change manager
Server-side branch control on production service connection | TBD | DevOps/security
Plan artifact retention, access and sensitivity classification | TBD | DevOps/security

Do not record storage keys, SAS tokens, OIDC tokens, client secrets, private keys, or Terraform state in this worksheet, Git, pipeline YAML, or plan output.

8. Monitoring, service levels, and operations

Input | Approved value / evidence reference | Accountable approver
Log Analytics workspace name, region and access mode | TBD | Observability
Workspace retention, data export, access and cost controls | TBD | Observability/compliance
Diagnostic categories and destination verification per resource type | TBD | Observability/DBA
CPU, physical data read, log write, storage, worker/session alert thresholds | TBD | DBA + SRE
Alert evaluation window, frequency, severity and action policy | TBD | SRE
Staffed action group receiver and test evidence | TBD | Operations
Built-in SQL operations workbook access and required fleet/database drilldowns | TBD | SRE/DBA
Selected diagnostic categories by resource type; sensitivity and ingestion-cost review | TBD | DBA + security + FinOps
SLI/SLO, user-facing availability/performance objectives | TBD | Service owner
Incident severity, escalation matrix and support contracts | TBD | Operations
Cost monitoring and budget threshold policy | TBD | FinOps
Routine access, policy, capacity, cost and alert review cadence | TBD | Service owner
Inventory/CMDB and ownership update process | TBD | Operations

9. Required cost estimate

Record estimate source, date, currency, term, region, assumptions and reviewer. Use the current Azure pricing calculator and organization-specific agreements; no numeric price is supplied by this repository.

Cost line | Quantity / assumptions | Monthly estimate | Reviewer / evidence
Primary elastic pools and vCore/license | TBD | TBD | TBD
Database storage and growth | TBD | TBD | TBD
Backup redundancy and retention | TBD | TBD | TBD
Log Analytics ingestion and retention | TBD | TBD | TBD
Azure Monitor alerting/action groups | TBD | TBD | TBD
Private endpoints, DNS, network transfer | TBD | TBD | TBD
Secondary region, replication and failover capacity | TBD | TBD | TBD
Support, nonproduction and test/restore resources | TBD | TBD | TBD
Total estimate, contingency and budget approval | TBD | TBD | TBD

10. Terraform input map

The following correspond to the current terraform/production module. Populate the actual reviewed configuration only after the preceding decisions are approved. The existing terraform.tfvars.example contains illustrative values and is not production input.

Terraform input | Source / validation before use
resource_group_name | Approved dedicated application resource group and owner.
location | Approved primary region; confirm every selected feature/SKU is available there.
sql_server_name | Globally unique lowercase letters/digits, 1-63 characters; confirm naming policy and availability.
sql_entra_admin_login | Approved Entra admin group display name.
sql_entra_admin_object_id | Verified, non-zero Entra group GUID in the target tenant.
private_endpoint_subnet_id | Exact network-approved subnet resource ID; permissions and address capacity verified.
private_dns_zone_id | Exact existing privatelink.database.windows.net zone resource ID; VNet link and DNS tested.
pools | Evidence-based workload map, tier/SKU/family/license/capacity/storage/per-database bounds/zone redundancy.
databases | Approved database names, pool assignments, storage ceilings, backup retention/redundancy.
workspace_name | Globally unique, policy-compliant workspace name and ownership.
workspace_retention_in_days | Approved range and cost/compliance review; default is not an approval.
operator_email | Staffed, tested, approved operational email.
pool_metric_alert_thresholds | Exactly six approved 0-100 thresholds for cpu_percent, physical_data_read_percent, log_write_percent, workers_percent, sessions_percent and storage_percent; tune against measured workload.
diagnostic_log_categories | Explicitly selected supported SQL log categories; the module intersects them with categories available on each resource. Review sensitive data, access and ingestion cost.
workspace_retention_in_days | Approved workspace retention with compliance and ingestion-cost basis.
operator_email | Staffed, monitored on-call receiver; test it in nonproduction and document acknowledgement/escalation.
tags | Organization-approved environment, service owner, cost center, data classification and required tags.

Review this table against variables.tf whenever module inputs change.
Terraform variable values must not contain credentials.

11. Approval record and final no-go conditions

Decision | Approver/name | Date | Evidence/change reference | Status
Service owner / scope | TBD | TBD | TBD | Pending
Capacity and SKU | TBD | TBD | TBD | Pending
Cost/budget | TBD | TBD | TBD | Pending
Network/DNS | TBD | TBD | TBD | Pending
Identity/RBAC/security | TBD | TBD | TBD | Pending
Backup/recovery/compliance | TBD | TBD | TBD | Pending
DevOps/state/pipeline controls | TBD | TBD | TBD | Pending
Operations/on-call/alerting | TBD | TBD | TBD | Pending
Nonproduction validation | TBD | TBD | TBD | Pending
Production change and exact plan approval | TBD | TBD | TBD | Pending

No-go: Any TBD in a required production input; absent evidence or owner; unapproved exception; unknown subscription/tenant; cost or quota uncertainty; failed private DNS/agent connectivity; missing protected approvals; unreviewed plan; missing restore evidence; or an action that creates an unapproved replacement, deletion, public path, or permission expansion.

12. External references to verify at implementation time

- Azure SQL elastic pool overview
- Azure SQL elastic pool management
- Azure SQL networking/private endpoints
- Azure SQL backup and restore
- Azure SQL failover groups
- AzureRM Terraform provider
- AzureRM backend authentication
- Azure DevOps workload identity federation
- Azure pricing calculator

Built-in observability and monitoring

The Terraform implementation includes a Log Analytics workspace, SQL

auditing, diagnostic settings on the logical server, master, every elastic pool and every managed database, an Azure Monitor workbook, a staffed email action group, and six saturation metric alerts per pool. Observability is managed with the database stack; it is not only an external recommendation.

This is a configurable baseline, not a tuned SLO. Thresholds, diagnostic categories, retention, access, notification routing, ingestion volume, and cost must be approved for the real environment. No Azure telemetry is collected while preparing or validating this repository.

Separate follow-up: Grafana and Prometheus

Grafana and Prometheus integration and dashboard setup are **not part of this Terraform deployment or the current observability deliverable**. They must be designed, estimated, approved, and implemented as a separate task. The Azure-native Log Analytics, Azure Monitor alerts, and workbook described here remain the current baseline; do not assume that a Grafana workspace, Prometheus server, exporter, scrape configuration, or Grafana dashboard is provisioned.

That separate task should confirm the monitoring platform and ownership, decide which Azure SQL signals come from Azure Monitor versus a supported exporter or other approved source, define private network and identity/access boundaries, select managed or self-hosted Grafana/Prometheus components, and document retention, availability, cost, alert routing, and operational ownership. Its acceptance should include validated data freshness and metric coverage, dashboard access controls, tested alert delivery, and an approved support runbook. Avoid adding a SQL exporter or credentials until its security and network design has been reviewed.

Provisioned monitoring surfaces

Surface | Terraform implementation | Coverage and operational use
Log Analytics | azurerm_log_analytics_workspace.sql | Dedicated destination; region and retention are environment inputs. Review access, ingestion, retention, query, and archive costs.
SQL auditing | azurerm_mssql_server_extended_auditing_policy.sql | Enables SQL audit log monitoring to the configured diagnostic destination; confirm audit policy, access, destination tables, and compliance retention in the target tenant.
Diagnostic settings | Server, master, every pool, and every database | Selects approved log categories and exports AllMetrics when supported. Category selection is intersected with categories exposed by each actual resource.
Metric alerts | azurerm_monitor_metric_alert.pool_saturation | Six rules for each pool; every rule routes to the configured action group.
On-call action group | azurerm_monitor_action_group.sql_oncall | Common alert schema email receiver; replace the example with a staffed, monitored and tested response destination.
Operations workbook | azurerm_application_insights_workbook.sql_operations | Log Analytics-backed trend chart and latest-sample table for the elastic-pool utilization metrics.
Terraform outputs | Workbook, workspace and alert resource IDs | Lets the release/handover process link directly to the provisioned observability resources.

Per-pool alert coverage

The Azure Monitor metric reference identifies these as elastic-pool platform metrics. Alert values are average percentages over a 5-minute window, evaluated every minute. Azure metric alerts use the REST metric identifier in the table; the workbook filters the corresponding AzureMetrics.MetricName display names. The example threshold map is deliberately illustrative and must be tuned to the workload by the DBA/SRE. The alert uses severity 2; change severity and response policy through an approved review.

Signal | Azure metric name | Why monitor it
CPU | cpu_percent (workbook display name: CPU percentage) | Compute saturation and sustained workload pressure.
Physical data reads | physical_data_read_percent (display name: Data IO percentage) | Data-I/O pressure. This is the Azure metric identifier; do not use data_io_percent.
Transaction log writes | log_write_percent (display name: Log IO percentage) | Log throughput pressure

and write/batch bottlenecks.
Workers | workers_percent (display name: Workers percentage) | Worker exhaustion and connection/request capacity pressure.
Sessions | sessions_percent (display name: Sessions percentage) | Session-capacity pressure and connection-pool behavior.
Storage | storage_percent (display name: Data space used percent) | Pool storage headroom; alert before the service/database ceiling is reached.

Thresholds are provided by pool_metric_alert_thresholds. Exactly these six metric keys are accepted, each with a value greater than 0 and at most 100. Current illustrative defaults are 80 percent for CPU, reads, log writes, workers and sessions, and 85 percent for storage; these values are not an approved production tuning. A single pool produces six alert rules.

Pool-level metrics can hide a hot individual database. Use the workbook, SQL diagnostic categories, Query Store, application telemetry, and workload-level analysis to investigate. Avoid automatic scaling based on one transient measurement; follow the capacity/change process.

Workbook

Terraform creates Azure SQL Elastic Pool Operations in Azure Monitor Workbooks, scoped to the configured Log Analytics workspace. It includes:

- A 24-hour trend view for CPU, physical data reads, log writes, workers, sessions, and storage by pool.
- The latest metric samples by pool resource ID and metric name.

The workbook is an operational starting view. Confirm that the workbook is visible to the intended operators, the workspace contains samples, the time range and resource filters are suitable for the workload, and queries execute with the operator's permissions. Add approved app-level latency, request correlation, deployment annotations, and SLO panels in a reviewed extension; those signals must be instrumented by the application and are not generated by the SQL infrastructure module.

Diagnostic categories and data handling

The default category selection is:

- SQLSecurityAuditEvents
- Errors, Deadlocks, and Timeouts
- DatabaseWaitStatistics
- QueryStoreRuntimeStatistics and QueryStoreWaitStatistics

Only categories both explicitly selected in diagnostic_log_categories and reported as available for a specific server/database/pool are enabled for that resource. This avoids assuming every SQL diagnostic category exists on every resource type. Confirm actual destinations, table schemas and event arrival in the target region/provider version; SQL resource-specific log tables may differ by destination mode. Audit and Query Store data can include sensitive workload metadata. Apply least-privilege workspace access, approved retention, data classification, and cost controls. Add categories only with an explicit security/compliance and ingestion-cost review.

Terraform exports logs and all available platform metrics only for the selected category/diagnostic destination. It does not export application traces, SQL query text as a substitute for Query Store controls, or customer data into this repository. A diagnostic setting is not proof that ingestion works: acceptance requires observing fresh representative events/metrics in Azure Monitor.

Alert response and acceptance

For each firing pool alert:

1. Identify the pool, metric, threshold, evaluation window and affected applications. Confirm user impact and the alert's current state before

taking action.

2. Open the operations workbook and compare the trend across the other pool signals. Identify whether pressure is pool-wide or driven by one database.

3. Check recent deployments, application retry/concurrency behavior, batch peaks, Query Store regressions, database-level limits, storage growth, and Azure Service Health.

4. Engage the DBA and application owner. For sustained risk, prepare a reviewed capacity or pool-placement change with cost, quota and recovery-region impact. Do not make an unreviewed scale or failover change during incident response.

5. Record alert/incident IDs, impact, samples, decision owner, action, acknowledgement, resolution and follow-up. Tune a threshold only through a documented DBA/SRE review with representative workload evidence.

Before service acceptance:

- Verify diagnostics on server, master, every pool and every database.
- Verify the intended categories and destination/table mode and observe new telemetry from each resource class.
- Confirm each pool has all six metric alerts with approved threshold, severity, 1-minute evaluation frequency, 5-minute window and the correct action group.
- Trigger notification through an approved nonproduction test and capture delivery, acknowledgement, escalation and closure evidence. Do not generate noisy/false production incidents to test routing.
- Open the workbook as an operator and verify meaningful trend and latest sample results; a blank panel is a failed check until explained.
- Verify retention, workspace access, audit access, cost reporting, and the incident/on-call owner.
- Confirm application teams own and test their request latency, failure rate, dependency health and trace correlation telemetry.

Cost, limits and operations

Workspace ingestion/retention and log-query costs depend on volume and current pricing; metric alert rules/workbook features may also have service-specific limits or charges. Estimate costs with real workload profiles. Set Azure budgets and usage alerts at the organization-approved subscription scope; the workspace retention setting is not a budget or an ingestion cap. Do not silently cap or discard required audit/security telemetry to reduce cost.

Alert count grows as six times the number of configured pools. Check Azure Monitor quotas, notification limits, receiver ownership, alert-rule naming, and action-group policies before scaling to a large pool fleet. If the fleet exceeds this starter's scope, design alert grouping, dynamic thresholds, action routing and workbook filters without losing per-pool accountability.

Review metric names, diagnostic categories, AzureRM schemas, workspace tables, regional support and pricing against the current Microsoft references linked below before a provider/service upgrade. Confirm signals remain live after every deployment, workspace change, pool change, and recovery exercise.

References

- Supported elastic-pool metrics
- Supported SQL database log categories
- Azure SQL metrics and alerts
- Azure Monitor Workbooks
- Azure Monitor diagnostic settings
- Azure Monitor pricing

Azure SQL elastic pool production deployment runbook

Status: Planning and deployment template only. No subscription, tenant, workload, network or approval values are associated with this repository. It is not deployed or certified production infrastructure. Do not deploy example or TBD values.

Service scope: Azure SQL Database elastic pools, vCore model, one primary region. Azure SQL Managed Instance and SQL Server on VMs are out of scope.

- Deployment entry point: terraform/production
- Delivery pipeline: azure-pipelines.yml
- Architecture: Reference architecture
- Readiness stages: Production readiness plan
- Environment decisions: Environment inputs worksheet
- Built-in monitoring: Observability and monitoring specification

1. Prerequisite readiness gates

Do not start a production apply until every item below is verified and the responsible owner has recorded the evidence in the change request. Microsoft-hosted agents can validate code, but private state access requires the configured deployment agent to reach the state account's private endpoint. The detailed owner/evidence worksheet is here; the stage gates and no-go conditions are here. Repository authoring and local credential-free checks do not create Azure resources. An authenticated plan or any later apply requires separately approved Azure access and may incur costs.

Gate | Required prerequisite and evidence | Owner
Scope and ownership | Approved service owner, database owner, operational/on-call contact, environment, data classification, cost center, and change ticket. | Service owner
Azure subscription | Correct tenant, subscription, management group/policy, resource group naming, approved primary region, and subscription/resource-group quotas. Confirm Microsoft.Sql, Microsoft.Network, Microsoft.Insights, and Microsoft.OperationalInsights providers are registered, deployer access is scoped, and budget is approved. | Cloud platform
Capacity and SKU | Database inventory; representative workload measurements; approved pool placement, vCore tier/hardware family, capacity, storage, per-DB bounds, zone redundancy, and cost estimate. Confirm SKU availability and regional quota. | DBA + workload owner
Identity | Entra administrator group/object ID and tenant; Microsoft Entra-only SQL sign-in; app managed identities or approved groups; no shared SQL admin password. | Identity + DBA
Network | Existing VNet, approved Private Endpoint subnet, address capacity, routing/NSG review, and egress to Azure Resource Manager and the private state backend. | Network
Private DNS | Existing privatelink.database.windows.net zone, correct VNet link, permissions for Private Endpoint DNS zone group, and tested resolution from workload subnets. | Network/DNS
State backend | Dedicated Azure Storage account/container, private endpoint and DNS, TLS 1.2+, public blob access disabled, shared-key access disabled, soft delete/versioning per policy, resource locks/backup where approved, and Azure Blob lease locking. | IaC platform
Azure DevOps | Project/repository, protected main, pull-request reviewers, pipeline authorization, approved self-hosted Linux deployment pool, and required Terraform installer extension/version. | DevOps
Workload identity | Azure Resource Manager service connection using workload identity federation (OIDC), explicit tenant/subscription, and least-privilege role assignments for the app resource group, subnet, DNS zone, and state container. | Cloud platform
Production approval | Protected sql-prod Azure DevOps Environment with named approvers/checks, restricted pipeline permissions, and an agreed rollback/stop authority. | Change manager
Observability | Approved Log Analytics workspace region/retention/access mode, metric alert thresholds, staffed action group recipient, incident routing, dashboards, and diagnostic retention/cost review. | SRE/observability
Security/compliance | Network and identity threat review, Azure Policy and Defender requirements, auditing/diagnostics, encryption requirements, data residency, vulnerability management, and deletion/restore controls. | Security/compliance
Resilience | Signed RPO/RTO, restore/failover design, secondary region and quota, database/pool mapping, application retry and DNS/listener plan, and scheduled test owner. | Service owner + DBA
Change safety | Nonproduction deployment and application test complete; reviewed Terraform diff/plan; backup/restore evidence; maintenance window; communications and escalation path. | Change approver

1.1 Resolve failed prerequisites before continuing

Symptom | Resolution; do not bypass the control
Wrong tenant/subscription, no subscription, or deploy denied | Stop. Use an approved identity, select the intended tenant and subscription, request scoped RBAC from the subscription owner, and rerun the identity checks. Do not widen permissions to Owner as a workaround.
SKU unavailable or quota insufficient | Have the capacity owner choose a supported tier/family/region or request quota through the subscription process. Recalculate cost and recovery capacity; update reviewed Terraform values. Do not reduce workload headroom without approval.
No measured inventory or workload baseline | Collect representative Azure SQL metrics and database ownership/criticality. Run a nonproduction load test where historical telemetry is inadequate. Do not infer pool size from database count alone.
No Entra administrator/group | Ask the Entra administrator to create or approve a managed operational group, verify its object ID in the correct tenant, and assign group membership through access review. Do not substitute a personal account or SQL password.
Private Endpoint subnet is invalid/full or lacks permissions | Network owner selects a supported subnet, reviews IP capacity/NSG/routes, and grants only the necessary subnet join/write permission. Re-read the exact resource ID; never create a duplicate/shared-network replacement without approval.
Private DNS lookup returns public/no address | Network/DNS owner verifies the exact private zone, VNet link, endpoint zone group, custom DNS forwarding, and resolver path. Test from the application subnet. Do not enable broad public SQL access to make the test pass.
State backend cannot be reached or authenticated | Check private agent routing/DNS, storage firewall/private endpoint, tenant/subscription, workload-identity federation, Storage Blob Data Contributor scoped to the state container, and AzureRM backend use_azuread_auth/OIDC settings. Do not use account keys or SAS tokens.
Azure DevOps cannot acquire OIDC token | Recreate/repair the service connection with workload identity federation, confirm the Azure DevOps issuer/subject/audience federation credential matches the connection, authorize the pipeline, and validate the correct tenant. Do not paste a client secret into YAML.
Pipeline identity lacks a required action | Use the failed Azure activity to identify the resource and permission. Have the owner add the narrow role at the target RG/subnet/DNS zone/container, then rerun a fresh plan. Do not grant Owner/Contributor at tenant or subscription scope just to silence an error.
Agent cannot reach Terraform downloads or Azure APIs | Have DevOps/network allow the approved artifact/provider registries and Azure endpoints, or use an approved internal mirror. Verify TLS inspection and agent clock/certificates. Do not disable certificate validation.
Pipeline has no protected environment/approvers | DevOps creates and secures the environment, restricts pipeline permissions, configures required approvers and branch checks, and tests a nonproduction gate. Do not remove approval to meet a deployment date.
Alert recipient/workspace policy is not approved | Observability team provides the staffed action-group target, workspace, region, access mode, retention and cost. Verify a test notification and alert ownership before accepting the service.
Restore/failover objective or evidence is missing | Service owner and DBA agree RPO/RTO and test a restore and regional recovery path with application owners. Record measured timings and data loss. No production go-live until the evidence matches the objective.
Terraform validate/plan fails | Read the first specific error; confirm Terraform version, provider lock file, required variables, region/SKU, IDs, permissions, backend reachability, and variable file. Fix the cause in a reviewed change. Do not suppress validation or apply a stale plan.

2. Operator and change controls

1. Identify the change owner, implementer, reviewer, approver, incident commander, application test contact, and rollback decision owner.
2. Attach the completed prerequisite evidence, capacity/cost review, security review, network approval, nonproduction test results, and recovery evidence to the change.
3. Freeze unrelated changes to the Terraform state key during the release. Terraform's Blob lease serializes state operations, but it does not replace a change freeze or review.
4. Review the source diff and the complete Terraform plan. Confirm all create/update/delete/replace actions and data-retention effects. If the plan is unexpected, stop; do not approve the production environment.
5. Use the Azure DevOps sql-prod Environment approval only after the exact pipeline plan artifact and change record have been reviewed.
6. Stop on any unplanned data-plane, network, identity, deletion, replacement, or destructive action. The starter pipeline has no destroy path.

3. Workstation and access prerequisites

3.1 Install and verify local tooling

Install approved, current versions of Git, Terraform CLI (same version as terraformVersion in azure-pipelines.yml), Node.js 18 or later, and Azure CLI. Install the Azure CLI Bicep extension only if validating the retained Bicep reference. Use organization-approved package sources and verify tool versions.

PowerShell:

```
git --version
terraform version
node --version
az version
```

If a command is not found, install it from the vendor/approved package repository, open a new terminal so PATH is refreshed, and rerun all four checks. If Terraform reports the wrong version, install the pinned version and align the pipeline version deliberately; regenerate and review .terraform.lock.hcl only when provider upgrades are approved.

3.2 Verify Azure identity and target

Use a named, approved operator identity; MFA and conditional access remain in force. Authenticate and verify the tenant before any change:

```
az login --tenant "<tenant-id>"
az account list --output table
az account set --subscription "<subscription-id>"
az account show --query "{tenant:tenantId,subscription:id,name:name,user:user.name}" --output json
```

Compare tenant and subscription IDs with the change record. If no account is returned, request access from the tenant/subscription owner. If the wrong subscription is active, stop and set the approved ID explicitly; do not rely on a remembered CLI context.

Local interactive login is for inspection and nonproduction testing. The production pipeline uses its federated identity and should not rely on a developer's CLI session.

4. Platform preparation

4.1 Network and DNS

Ask the network owner to provide and verify:

- VNet and subnet resource IDs in the SQL server's region or an approved supported topology. The endpoint subnet must support Private Endpoints and have available IP addresses.
- Required subnet policy, route table, NSG, DNS forwarding, and firewall controls for the endpoint and applications.
- The existing privatelink.database.windows.net Private DNS zone ID and a link to each approved client VNet. The Terraform endpoint zone group attaches the private endpoint to the provided zone; it does not create the zone/link.
- Private DNS forwarding/resolver behavior for corporate and build networks. From an approved workload host, verify the server FQDN resolves to the endpoint's private address and that TCP 1433 is reachable only where authorized.
- A self-hosted Azure DevOps Linux agent with outbound Azure Resource Manager access and network/DNS access to the private state backend. For SQL application smoke tests, use a separate approved workload runner inside the application network.

Do not enable SQL public network access or add an unrestricted firewall rule to compensate for missing routing, DNS, or build-agent connectivity.

4.2 Terraform state backend

Create the state resource group, dedicated StorageV2 account and blob container before running the application Terraform. Use the organization's approved region/redundancy, private endpoint, private DNS zone privatelink.blob.core.windows.net and VNet link, network restrictions, TLS 1.2 minimum, disabled public blob access, disabled shared-key access, Microsoft Entra data-plane authorization, soft delete/versioning, and state backup/retention policy. Enable Blob versioning and deletion recovery as required by policy. Restrict state and historical versions because Terraform state can contain sensitive resource metadata.

For a dedicated state resource group, the following is an example bootstrap sequence. Run it only with the approved platform/bootstrap identity and adapt the region, redundancy, names, tags, and retention to organizational policy. The Storage account is created with public network access disabled; the network team must attach its private endpoint and DNS before any blob data-plane operation. Do not temporarily open public access to get around a missing private route.

```
$stateResourceGroup = "rg-tfstate-prod"
$stateAccount = "<globally-unique-state-storage-name>"
$stateLocation = "<approved-state-region>"
$stateContainer = "tfstate"
az group create --name $stateResourceGroup --location $stateLocation
az storage account create `
--name $stateAccount `
--resource-group $stateResourceGroup `
--location $stateLocation `
--sku Standard_GRS `
--kind StorageV2 `
--https-only true `
--min-tls-version TLS1_2 `
--allow-blob-public-access false `
--allow-shared-key-access false `
--public-network-access Disabled
```

After the network team confirms the Storage private endpoint and DNS are reachable from the private deployment agent, enable recovery controls and create the container using Entra data-plane authentication:

```
$subscriptionId = az account show --query id --output tsv
$storageScope =
"/subscriptions/$subscriptionId/resourceGroups/$stateResourceGroup/providers/Microsoft.Storage/storageAccounts
/$stateAccount"
# A role administrator assigns this to the approved bootstrap identity.
az role assignment create `
--assignee-object-id "<bootstrap-principal-object-id>" `
--assignee-principal-type User `
--role "Storage Blob Data Contributor" `
--scope $storageScope
# Allow for RBAC propagation before using the blob data plane.
az storage account blob-service-properties update `
--account-name $stateAccount `
--resource-group $stateResourceGroup `
--enable-versioning true `
--enable-delete-retention true `
--delete-retention-days 30 `
--enable-container-delete-retention true `
--container-delete-retention-days 30
az storage container create `
--account-name $stateAccount `
--name $stateContainer `
--auth-mode login
az storage container show `
--account-name $stateAccount `
--name $stateContainer `
--auth-mode login `
--output table
```

The bootstrap identity needs approved management-plane rights to create the resource group/account and storage data-plane rights to create the container. Have the role administrator grant the pipeline identity **Storage Blob Data Contributor** at the existing container scope, not by sharing a key:

```
$subscriptionId = az account show --query id --output tsv
$containerScope = "$storageScope/blobServices/default/containers/$stateContainer"
az role assignment create `
--assignee-object-id "<azure-devops-service-principal-object-id>" `
--assignee-principal-type ServicePrincipal `
--role "Storage Blob Data Contributor" `
--scope $containerScope
```

Follow the platform's lock, monitoring, soft-delete retention, and state restoration policies. If provider registration is missing, ask the subscription owner to approve/register the listed providers; do not use broad subscription permissions as a workaround. Test az storage container show from the deployment agent, then enter the storage names and stable key in the protected Azure DevOps variable group.

Use a dedicated, stable key such as azure-sql/prod.tfstate. The AzureRM backend uses blob leases for state locking. Grant the pipeline identity Storage Blob Data Contributor at the state container (or the narrowest supported scope), and authorize state-management actions separately. Never commit state, plan files, access keys, SAS tokens, credentials, or state backups.

If the storage account is private-only, the plan and apply agent must be able to resolve and reach its private endpoint before pipeline use. Test access from that agent; an Azure-hosted Microsoft-managed runner does not automatically have access to a private VNet. Use the organization's private deployment pool.

4.3 Identity and Azure DevOps

1. Create an Azure DevOps project/repository and protect main: require reviewed pull requests, build validation, no direct pushes, and restricted pipeline-edit permissions.
2. Install the approved Microsoft DevLabs Terraform extension to provide TerraformInstaller@1. Pin the tested Terraform CLI version in azure-pipelines.yml; maintain the Azure CLI task on the Microsoft-hosted task catalog.
3. Create an Azure Resource Manager service connection with **workload identity federation**, not a client secret. Use the exact tenant and subscription. Configure and validate the generated federation credential; authorize only this pipeline. Add a server-side Branch control check to the service connection that allows only protected refs/heads/main and fails closed for unknown protection status. Do not rely on YAML stage conditions alone to protect the connection from pull-request code.
4. Assign the service identity only the resource-group rights needed to create SQL, monitoring, and private endpoint resources, plus the narrowly scoped subnet and Private DNS permissions required for endpoint creation. Grant state Blob data access on the state container. Do not grant tenant-wide directory roles or subscription Owner for routine deployment.
5. Create an approved, private-connected Linux deployment agent pool. Install Git, Bash, Azure CLI, the pinned Terraform CLI or Terraform extension support, and required CA certificates. Keep agent patches/current images and restrict who can administer the pool.
6. Create the Azure DevOps variable group named sql-elastic-pool-prod; add these non-secret values and authorize only the deployment pipeline:

Variable | Example/meaning
azureServiceConnection | Exact workload-identity service connection name.
deploymentAgentPool | Private Linux agent pool name.
azureSubscriptionId | Approved subscription GUID.
tenantId | Approved Microsoft Entra tenant GUID.
tfstateResourceGroup | State storage resource group name.
tfstateStorageAccount | Globally unique state StorageV2 account name.
tfstateContainer | Existing private blob container name.
tfstateKey | Stable state key, e.g. azure-sql/prod.tfstate.
tfVarsFile | environments/prod.tfvars (relative to terraform/production).

These values are identifiers, not credentials. Do not add client secrets, storage keys, SQL passwords, SAS tokens, or OIDC tokens. Do not make the variable group broadly accessible.

7. Create the Azure DevOps Environment sql-prod, restrict pipeline access, and configure required named approvers/checks, branch control, and any organization-required change-window checks. Verify that the approval is evaluated before apply.

8. Configure protected repository policies and pipeline resource permissions. A pull request can run offline validation but must not receive the production service connection or state access. Verify the service connection's server-side Branch control check rejects a test run from a feature branch; YAML conditions alone are not an authorization boundary.

9. In Azure DevOps, create a pipeline from this repository and select /azure-pipelines.yml. Authorize the variable group, workload-identity service connection, and private deployment-agent pool for this pipeline only. Run the initial PR validation first; the main pipeline must show a saved plan and stop for the protected sql-prod approval before any apply.

4.4 Workload resource group and Azure providers

Terraform expects the dedicated workload resource group to exist. The platform team creates it under the approved landing-zone policy, tags, budget, locks, and region. If policy allows the bootstrap operator to create it directly:

```
$resourceGroup = "rg-sql-prod"
$location = "<approved-sql-region>"
az group create `
--name $resourceGroup `
--location $location `
--tags environment=production serviceOwner="<owner>" costCenter="<cost-center>"
dataClassification="<classification>"
az group show --name $resourceGroup --output table
```

Replace every placeholder and confirm the policy assignments, location, ownership tags, budget, and lock behavior with the landing-zone owner. Do not create a second group if the platform team has already provided one.

Check that required resource providers are registered in the target subscription:

```
$namespaces = @(
"Microsoft.Sql",
"Microsoft.Network",
"Microsoft.Insights",
"Microsoft.OperationalInsights",
"Microsoft.Storage"
)
foreach ($namespace in $namespaces) {
az provider show --namespace $namespace --query registrationState --output tsv
}
```

If any provider is not Registered, have the subscription owner review and register that namespace (az provider register --namespace <namespace>), then wait for registration to complete before planning. The Azure DevOps identity needs the approved resource-group permissions for SQL/monitoring resources and separate narrow subnet/Private DNS permissions for the Private Endpoint. It must not receive permission to create role assignments; the platform identity pre-assigns the required roles.

5. Capacity, configuration, and code review

5.1 Select pool and database configuration

Start from the inventory and measurements, not the example values. Set:

- location, resource group, globally unique SQL server name, Entra administrator login/object ID, and approved resource tags. Provider tenant and subscription are taken from the verified federated/CLI identity.
- private_endpoint_subnet_id and private_dns_zone_id from the network

owner.

- pools: one entry per workload-aligned pool; supported vCore SKU/tier, capacity, hardware family, license model, storage, per-database minimum and maximum, and zone redundancy.
- databases: database-to-pool assignment, storage limit and short-term-retention days, plus approved backup storage redundancy (Geo, GeoZone, Local, or Zone).
- workspace_retention_in_days, action group email, selected diagnostic log categories, and measured thresholds for pool CPU, physical data reads, transaction log writes, workers, sessions and storage.

The values in terraform.tfvars.example are illustrative only; the environment-specific environments/prod.tfvars.example is a non-deployable TBD worksheet. Confirm that each SKU/family and feature is available in the selected region and supported for every database. Validate aggregate pool capacity, per-database limits, storage and workload concurrency. BasePrice requires approved Azure Hybrid Benefit eligibility; zone-redundant vCore pools require a supported BusinessCritical SKU/region. Get written approval for exceptions.

5.2 Create the production variable file

Create the environment directory and copy the example:

```
Copy-Item terraform/production/environments/prod.tfvars.example `
terraform/production/environments/prod.tfvars
```

The tracked file contains TBD and intentionally invalid pool/tag values. Replace all of them with reviewed, non-secret production configuration only after the readiness gates and worksheet are approved. Do not create a valid production configuration by guessing.
Do not add passwords, keys, tokens, private certificates, or credentials; Entra-only authentication eliminates the need for a SQL admin password. Terraform configuration values can expose infrastructure identifiers. Review repository access and secret-scanning rules before committing the environment file. The production pipeline intentionally fails until the approved file exists.

The repository includes terraform/production/.terraform.lock.hcl with verified checksums for the Linux pipeline agent and Windows local validation. From an approved network, initialize Terraform using the lock file and verify it does not change. For an approved provider upgrade, review release notes, regenerate checksums for every deployment platform in a controlled change, validate in nonproduction, and commit the reviewed lock-file update. Do not remove the lock file to work around a provider error.

6. Validate and plan

6.1 Credential-free checks

Run from the repository root:

```
terraform -chdir=terraform/production fmt -check -recursive
terraform -chdir=terraform/production init -backend=false
terraform -chdir=terraform/production validate
node scripts/build-pdf.mjs
git diff --check
```

These checks need no Azure credentials, but provider initialization requires approved network access to the Terraform Registry or the organization's provider mirror.

If fmt fails, run terraform -chdir=terraform/production fmt -recursive, review the result, and rerun all checks. If provider installation fails, verify the approved provider registry/mirror and TLS path; do not disable TLS checks. If validation reports a missing variable or invalid region/SKU, repair the

reviewed configuration and rerun. Do not proceed with a failed check.

6.2 Authenticated nonproduction plan

First deploy and test in a separate nonproduction resource group, using its own reviewed variable file (copy the example to environments/nonprod.tfvars), backend key, and protected identity. Authenticate through an approved identity and verify the target subscription. Run terraform init against the dedicated remote backend. With Azure CLI authentication and the nonproduction values substituted, initialize the AzureRM backend as follows:

```
$env:ARM_SUBSCRIPTION_ID = az account show --query id --output tsv
$env:ARM_TENANT_ID = az account show --query tenantId --output tsv
# Confirm both values match the approved tenant and subscription in the change.
terraform -chdir=terraform/production init -reconfigure `
-backend-config="resource_group_name=<state-resource-group>" `
-backend-config="storage_account_name=<state-storage-account>" `
-backend-config="container_name=<state-container>" `
-backend-config="key=azure-sql/nonprod.tfstate" `
-backend-config="subscription_id=$env:ARM_SUBSCRIPTION_ID" `
-backend-config="tenant_id=$env:ARM_TENANT_ID" `
-backend-config="use_azuread_auth=true"
```

Then create the saved plan:

```
terraform -chdir=terraform/production plan `
-input=false `
-lock-timeout=5m `
-parallelism=5 `
-var-file=environments/nonprod.tfvars `
-out=nonprod.tfplan
```

Review the plan, test DNS/private connectivity, application authentication, diagnostic ingestion, action-group delivery, and realistic workload. Treat plan files as sensitive; store only in an access-controlled pipeline artifact or delete a local file after approved use. Do not apply a plan produced from a different commit, backend, variable file, identity, or environment.
The pipeline limits apply/plan concurrency to five operations to reduce resource-provider throttling during fleet-scale deployment. Change that limit only after observing provider quotas and representative deployment timing.

6.3 Production plan in Azure DevOps

The pipeline runs credential-free formatting/validation for pull requests. After reviewed code merges to protected main, it uses the federated service connection to initialize the AzureRM backend and creates an immutable tfplan artifact. Reviewers compare the artifact to the merged Terraform, variable file, ticket, resource IDs, cost, and expected actions. An unexpected replacement or deletion blocks approval.

The production plan includes the full parameter values in Terraform's execution context. Limit artifact access/retention to authorized operators. Never publish it as a public artifact or attach it to an unrestricted work item. The apply stage downloads and applies the exact saved artifact; it does not silently re-plan.

7. Production apply

1. Confirm prerequisite gates, nonproduction acceptance, active change window, reviewer sign-off, exact main commit, plan artifact, state key, and target subscription.
2. Verify the sql-prod environment displays the required approvers and checks.
3. Approve only when planned resources, replacements, action group, SKU, private endpoint, diagnostic settings, retention, and estimated costs match the approved change.
4. The deployment job applies the saved plan. Watch the Azure DevOps job and Azure Activity Log. Do not run a second concurrent pipeline on the same key.

5. Stop on apply errors or drift. Preserve logs and plan identifiers; diagnose the failing Azure API, permission, policy, quota, provider, or network issue. Fix the cause through a reviewed commit and generate a fresh plan.

6. Do not hand-edit Terraform-managed resources in the portal. For an approved emergency change, record it, notify the IaC owner, and reconcile configuration/state through a reviewed Terraform plan as soon as possible.

8. Post-deployment acceptance checklist

Complete and attach evidence to the change record:

- [ ] SQL logical server exists in the approved subscription, region and resource group; TLS minimum, Entra-only authentication, public network access, identity, tags, and Policy compliance match the design.
- [ ] Entra administrator object ID and tenant are correct. An authorized operator signs in with Entra authentication; no SQL administrator password has been created or shared.
- [ ] Every pool has the approved SKU, family, capacity, storage, per-database limits, and zone-redundancy setting. Every database is in the intended pool with correct maximum storage and retention.
- [ ] Private Endpoint is approved/connected. SQL FQDN resolves to the private address from each intended workload VNet and does not create an unintended public application path.
- [ ] From an approved workload host, TLS certificate validation succeeds and the application connects on TCP 1433 using its authorized identity. Denied connections from an unauthorized network are also verified.
- [ ] Diagnostic settings are enabled on the logical server, every elastic pool, every database and master. The selected diagnostic categories and destination are correct; fresh logs/metrics arrive in the approved workspace.
- [ ] All six saturation metric alerts target every pool: CPU, physical data reads (physical_data_read_percent), log writes, workers, sessions and storage. Thresholds, severity, window, frequency and action group match the approved observability worksheet.
- [ ] The built-in Azure Monitor workbook opens for the intended SRE/DBA groups and returns current trend and latest-sample results. A blank or unauthorized panel is investigated rather than accepted.
- [ ] Alert routing is tested in nonproduction; delivery, acknowledgement and escalation are evidenced. Do not generate a noisy production alert solely for a routing test.
- [ ] Cost tags, budget alerts, Defender, auditing, retention, access reviews, and policy assignments meet the service's security/compliance baseline.
- [ ] Point-in-time restore is exercised into an isolated target and timed. Record the restore point, achieved RPO/RTO, owner, and cleanup approval.
- [ ] Regional recovery/failover is tested according to the approved plan; confirm secondary capacity, listener/DNS behavior, application reconnect, and data loss against RPO/RTO.
- [ ] SRE/DBA has dashboards, runbooks, on-call contacts, alert ownership, capacity review schedule, patch/change process, and escalation details.
- [ ] Terraform state is healthy, the pipeline is green, the artifact is access-restricted, and the next approved change can produce a no-op plan.

Do not onboard business data or declare production service until any failed acceptance item is resolved or formally risk-accepted by the accountable owner.

9. Operational monitoring and scaling

Review the built-in [Azure Monitor workbook and six per-pool saturation alerts](observability-monitoring.md): CPU, physical data reads, log writes, workers, sessions and storage, alongside latency, errors and database-level hotspots. Aggregate metrics can conceal a single overloaded database. Use Azure SQL Query Store/approved diagnostic queries to identify workload

regressions without exporting data outside approved boundaries.

For sustained saturation, confirm the affected databases and correlated peak, check application/query changes and limits, evaluate pool isolation or right-sizing, calculate cost and recovery-region impact, test in nonproduction, then deploy via a reviewed plan. For storage pressure, investigate database growth and retention before resizing. Do not scale automatically from one transient metric or change pool configuration during an incident without the incident commander/change authority.

Review pool placement, database ownership, cost, alert thresholds, quota, and secondary capacity at the agreed cadence and after workload or business changes.

10. Incident and recovery actions

10.1 High utilization or widespread connection errors

1. Incident commander confirms impact, affected pool/database, scope, and severity; notify database and application owners.
2. Check Azure Service Health, resource health, pool/database metrics, diagnostic logs, Query Store, recent releases, connection throttling, workers, storage, and network/DNS from the workload subnet.
3. Mitigate through an approved application rollback, workload throttle, workload isolation, or pre-approved capacity change. Record cost and failover consequences. Use a fresh reviewed pipeline plan for IaC changes.
4. Verify recovery using application SLOs and alert recovery; preserve timeline, evidence, and follow-up actions.

10.2 Suspected data loss, corruption, or regional outage

1. Declare the incident, stop unsafe writes if directed, preserve evidence, and consult the service owner/DBA recovery decision authority.
2. Choose point-in-time restore, geo-restore, failover group, or geo-replication according to the approved data-loss and RTO decision. Confirm destination, credentials, capacity/quota, DNS/listener, encryption, and application dependencies.
3. Test recovered data and application invariants with the owner before cutover. Record actual restore point, data loss, duration, approvals, and endpoint change.
4. Restore the primary design only under a reviewed change plan. Reconcile Terraform and state without importing or replacing resources ad hoc.

Do not run failover, restore, import, state removal, or resource deletion from this template without the incident commander and authorized DBA approving the specific recovery action.

11. Change, teardown, and state safety

All infrastructure changes go through a reviewed PR, successful CI validation, protected branch, production plan review, and sql-prod approval. Refresh Terraform CLI/providers only through a reviewed dependency update and lock-file change. Keep state backups/version history and test restoration procedures.

There is no production destroy stage. Retiring a service requires a separate approved data-retention and backup/export plan, application cutover, owner sign-off, and a reviewed resource inventory. Do not delete or reinitialize state, unlock a lease, remove a resource from configuration, or use terraform state rm as an incident shortcut. Escalate lock/state recovery to the IaC platform owner and preserve forensic evidence.

12. Database schema and data onboarding

The Terraform database resource creates an empty pooled database. It does not

create schemas, principals, application objects, or import/migrate business data. Treat database initialization and migration as a separate, versioned application release:

1. Confirm the database owner, source/target inventory, data classification, approved migration method, maintenance window, downtime/consistency requirements, and rollback/restore point.
2. Use the approved migration tool and a private network-connected migration runner. Authenticate with a managed identity or approved Entra principal; grant only the database permissions required for the migration and remove temporary grants after verification.
3. Version schema and seed/reference data changes in the application repository. Validate scripts against the target Azure SQL engine, selected service tier, collation, compatibility level, and application release.
4. Prefer backward-compatible expand/migrate/contract releases: add compatible objects first, deploy application code that can use both schemas, backfill in observable batches, validate row counts and business invariants, then remove obsolete objects only in a separately approved later release.
5. Test the full migration and recovery procedure against representative nonproduction data. Review lock duration, transaction/log growth, throttling, retry/idempotency behavior, and expected data loss/downtime.
6. On production, verify endpoint/private DNS and Entra access, take/confirm the required restore point, run the approved migration, validate application health and data invariants with the owner, and record timing/evidence.
7. If validation fails, stop writes/cutover as agreed and follow the approved restore or forward-fix decision. Do not assume a schema rollback reverses data changes, and do not run destructive rollback SQL without owner approval.

The platform pipeline deliberately has no SQL-password variable or application data migration command. Keep schema deployment separately reviewed and credentialed.

13. Ownership and review cadence

The service owner reviews objectives, risk acceptance, cost, and recovery. DBAs own pool/database placement, sizing, backups, and SQL health. Network owners own Private Link, DNS, routing, and address capacity. Identity/security owners approve authentication, federated identity, RBAC, policy, audit, and data controls. DevOps owns agent, pipeline, artifact, approvals, and branch protection. SRE owns alert response, dashboards, incident process, and drills.

Review this runbook after Azure service/provider changes, material architecture or compliance changes, every recovery exercise, and at least annually. Verify the current Microsoft provider/API guidance before production upgrades.